

# **Estruturas de Dados II**

## **COMPARAÇÃO DE ÁRVORES BALANCEADAS: ANÁLISE DE CUSTOS COMPUTACIONAIS POR IMPLEMENTAÇÃO DE ÁRVORES AVL, B E RUBRO-NEGRAS**

**Alunos:**

**Luciano Kohler da Silva**

**Luiza Mannes Silveira**

**Gabriel Oliveira Willemann**

**Matheus Eduardo Gomes de Santana**

**Professor:**

**Allan Rodrigo Leite**

**Joinville - 2025**

## SUMÁRIO

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>INTRODUÇÃO.....</b>                 | <b>3</b> |
| <b>2</b> | <b>OBJETIVOS.....</b>                  | <b>4</b> |
| <b>3</b> | <b>METODOLOGIA.....</b>                | <b>4</b> |
|          | 3.1 Estrutura do projeto.....          | 4        |
|          | 3.2 Procedimento experimental.....     | 4        |
| <b>4</b> | <b>RESULTADOS E DISCUSSÃO.....</b>     | <b>5</b> |
|          | 4.1 Esforço de Inserção.....           | 5        |
|          | 4.2 Esforço de Remoção.....            | 6        |
| <b>5</b> | <b>CONCLUSÃO.....</b>                  | <b>6</b> |
| <b>6</b> | <b>REFERÊNCIAS BIBLIOGRÁFICAS.....</b> | <b>7</b> |

## 1 INTRODUÇÃO

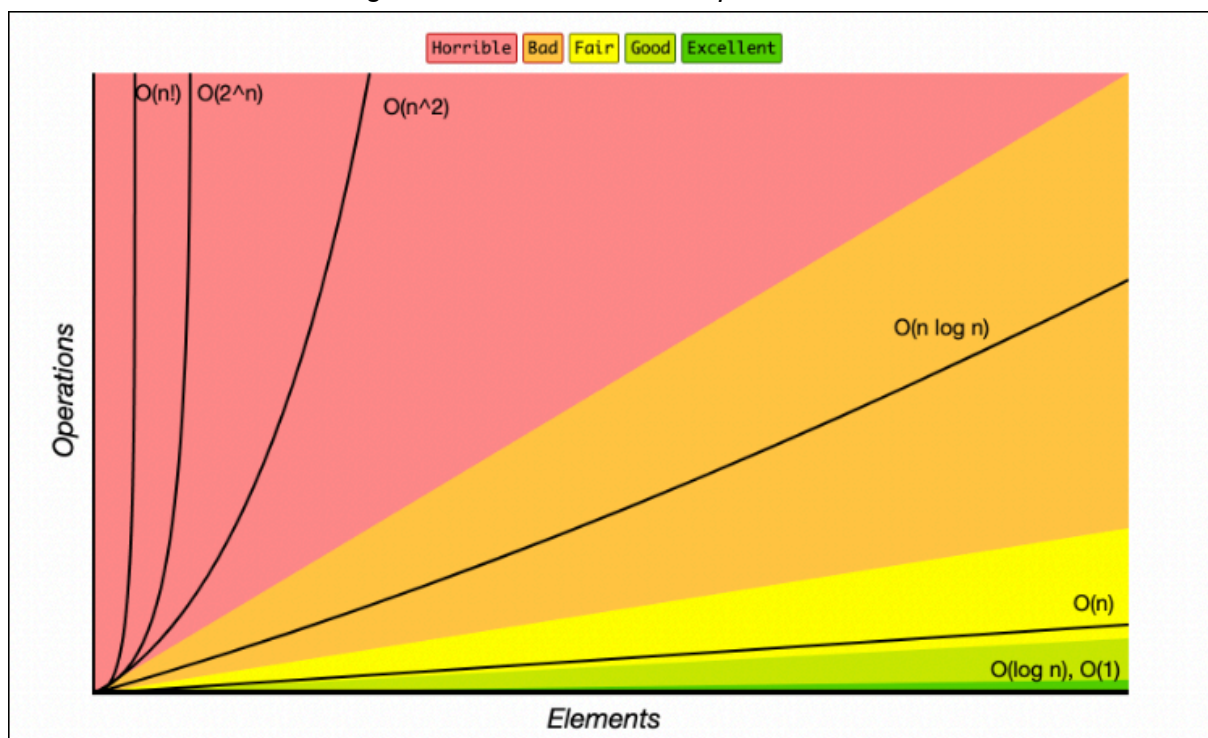
Na computação, uma **árvore balanceada** é uma estrutura de dados que armazena informações por meio de **nodos**, unidades de armazenamento que, além de guardar uma informação, armazenam também nodos **filhos** que, por ser balanceada, obrigatoriamente seguem um padrão que permite ordenar os dados dentro da estrutura para, assim, armazenar dados de forma muito otimizada em comparação à uma armazenagem comum de dados.

Das inúmeras ramificações da estrutura de árvore balanceada presentes na área da programação, 3 se destacam:

- **Árvore AVL**, que usa de rotações e fatores de balanceamento para manter sua estrutura ótima
- **Árvore Rubro-negra**, que usa de coloração de nodos para decidir a melhor formatação da árvore
- **Árvore B**, que em complemento com o balanceamento, usa uma estrutura secundária de dados para armazenar mais de uma chave por nodo, a quantidade de dados salvos é regida pela **ordem** da árvore

O objetivo principal das árvores balanceadas é o de reduzir o *custo computacional* do programa, ou seja, a quantidade de recursos de tempo e processamento necessários para cumprir os requisitos do programa, veja à seguir, uma representação gráfica dos diferentes custos computacionais denotada de *Big O Graphic*:

Figura 1: Gráfico de custos operacionais



Fonte: FreeCodeCamp.org

Como é possível ver, por exemplo, uma complexidade constante ( $O(1)$ ), ou logarítmica ( $O(\log n)$ ) são ideais e devem ser buscados na hora da implementação de softwares no geral, enquanto complexidades exponenciais ( $O(2^n)$ ), ou fatoriais ( $O(n!)$ ) são

um veneno para o software, pois indicam uma crescente falta de desempenho no sistema conforme o número de operações (n) aumentam.

## 2 OBJETIVOS

Visto essas informações, este relatório procura comparar o **custo computacional** das árvores para o processo de inserção e deleção de elementos aleatorizados e de tamanho variado, mais especificamente, serão gerados 10 vetores variando de 1000 a 10000 elementos completamente aleatórios, assim evitando a ocorrência de piores ou melhores casos para árvores específicas.

Será gerado também, gráficos comparativos incluindo a quantidade de iterações que cada árvore levou para operar sobre o conjunto de N elementos para a visualização das diferenças de desempenho.

## 3 METODOLOGIA

### 3.1 Estrutura do projeto

O projeto feito em linguagem C possui 4 elementos-chave:

- O arquivo **main.c**, responsável por aglomerar todas as estruturas, executá-las e armazenar os resultados das inserções/remoções
- A pasta **estruturas**, armazenando cada uma das estruturas (AVL, B de ordem 1, B de ordem 5, B de ordem 10 e Rubro-Negra)
- Um cabeçalho **header.h**, incluindo a definição de todas as estruturas do código para importação mais modularizada
- A pasta **plot**, responsável por armazenar os dados gerados e os códigos em Python responsáveis por gerar os gráficos de média de iterações por conjunto

### 3.2 Procedimento experimental

Todo o código é executado à partir da compilação do código **main.c**, que inclui as estruturas, faz o processamento de dados e por fim imprime o desempenho de cada uma das árvores para certos conjuntos de dados específicos, o funcionamento segue uma sequência de passos que pode ser descrita como:

1. **Declaração de variáveis:** Onde são declaradas as variáveis necessárias para a execução das árvores e a estrutura principal do programa que irá salvar as iterações que cada árvore fez para concluir a inserção completa do conjunto de dados.
2. **Geração de valores aleatórios:** Aqui, são gerados N valores aleatórios por meio da função **rand()**, que são então armazenados em um vetor denominado **conjValoresAleatorios[N]**, que ao longo das iterações, tem seu tamanho aumentado e seus valores gerados novamente.
3. **Inserção e Deleção em cada Árvore:** A parte mais vital do programa, onde, para cada uma das estruturas analisadas (AVL, B1, B5, B10 e Rubro-Negra), são inseridos todos os valores gerados no passo anterior (salvando as iterações no processo) e, em seguida, é realizada a remoção dos valores seguindo a mesma ordem do vetor original\*, também salvando a quantidade de iterações necessárias para análise futura

*\*Nota: Como as estruturas são balanceadas, a remoção pela ordem original do vetor aleatório não impactam com o enviesamento de dados, pois as árvores dissolvem o aspecto aleatório do conjunto de dados original quando suas rotações/colorações/promoções são realizadas.*

4. **Impressão e salvamento dos resultados:** Por fim, os resultados de inserção e remoção são mostrados na tela em forma de tabela formatada em texto, indicando a quantidade de iterações que cada árvore tomou para a inserção/remoção de N valores. Além disso, os dados são manualmente salvos na pasta **plotting** em documentos **.csv**.

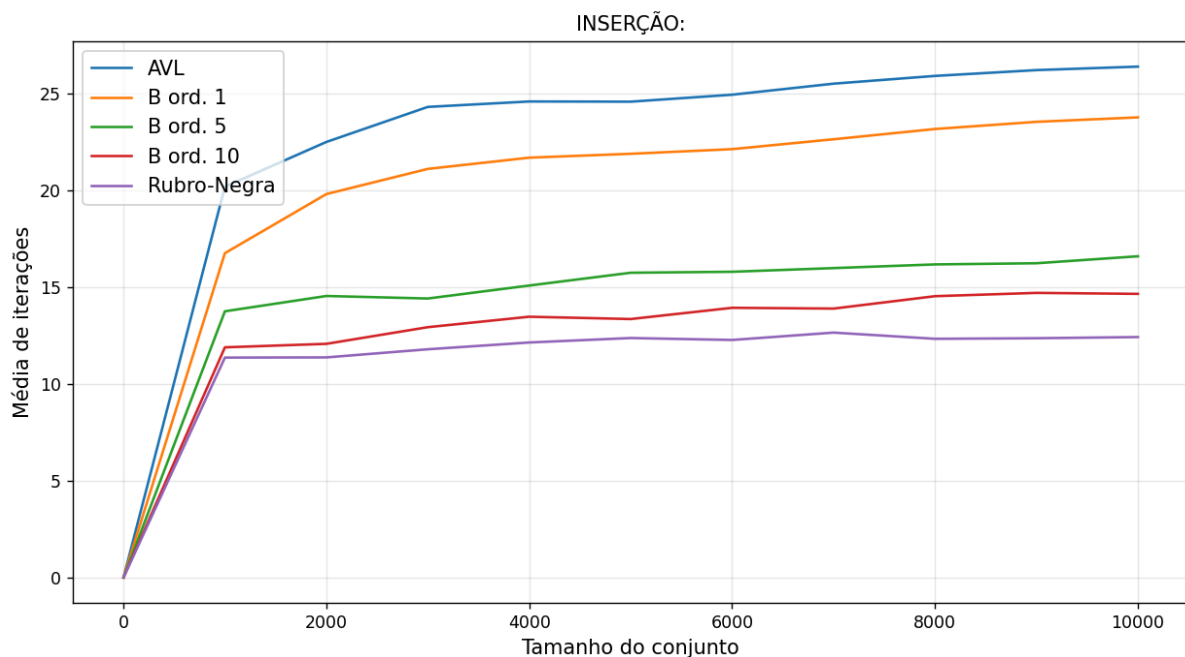
Os passos 2 a 4 então, são repetidos mais 9 vezes, uma vez para cada conjunto de dados novo, iniciando em 1000 valores e finalizando em 10000.

## 4 RESULTADOS E DISCUSSÃO

Gerando dados pela execução repetida do código, aglomerando os dados e gerando uma média de iterações e colocando esses dados em uma amostra visual, é possível, por fim, encontrar esses tipos de dados:

### 4.1 Esforço de Inserção

Figura 2: Gráfico de custo computacional de inserção

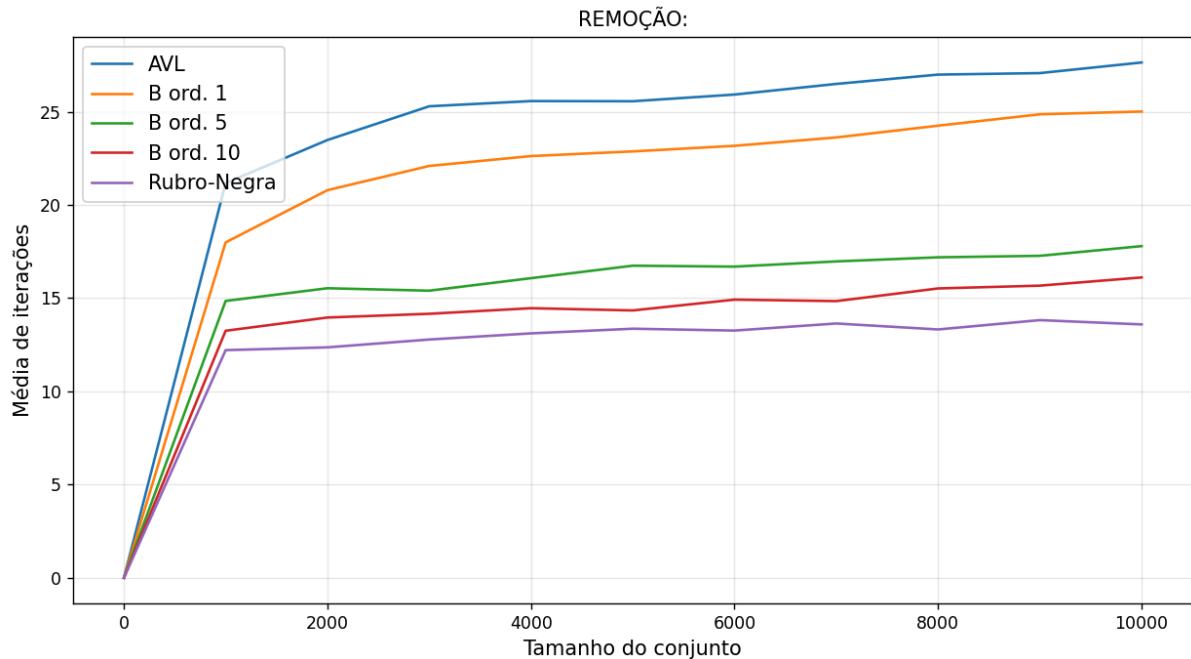


Ordenando tudo em um ranqueamento, temos então que:

1. A árvore AVL toma a posição de mais custosa para todos os conjuntos de dados
2. As árvores B seguem logo atrás com desempenhos variados, mas indicando que uma ordem maior tende a custar menos
3. A árvore Rubro-Negra toma o pódio como a árvore mais eficaz, apresentando uma curva muito baixa comparada aos seus oponentes.

## 4.2 Esforço de Remoção

Figura 3: Gráfico de custo computacional de remoção



Similarmente à inserção, o pódio das árvores não mudou, e a árvore Rubro-Negra segue como a árvore ótima para uma implementação eficiente.

## 5 CONCLUSÃO

Após toda a análise, é possível gerar diversas conclusões em cima dos dados:

- É possível notar o comportamento logarítmico da média das iterações de cada árvore, esse comportamento vem da característica principal das árvores balanceadas, onde a altura máxima que essa árvore alcança é  $\log(n)$ . Isso permite com que a quantidade máxima de nodos percorridos para uma inserção ou remoção seja, também,  $\log(n)$ .
- Pela análise dos dados, é possível inferir que a árvore Rubro-Negra, mesmo que envolvendo uma lógica muito mais complexa do que as outras, recompensa esse fato por ser a árvore de melhor desempenho, tendo uma média de iterações menor do que as outras árvores comparadas. Isso ocorre pois, além de envolver **rotações** para o balanceamento da árvore, a mesma também usa do método da **coloração** de nodos para agilizar o processo de busca na estrutura.
- Também é possível notar analisando o comportamento das árvores B que uma implementação de árvore B só compensa seu custo computacional se sua ordem for razoavelmente grande, por volta de 5 ou 10, isso ocorre pois uma ordem baixa ocasiona em mais *overflows* e *underflows*, necessitando de mais reordenações para manter a estrutura ideal da árvore.

- Por fim, é possível ver que a árvore AVL (mesmo com suas técnicas próprias de balanceamento) fica para trás quando comparada com o desempenho das outras árvores, isso se dá pelo fato de que a sua lógica de rotações é muito mais simples do que quando comparada à coloração dos nodos da árvore Rubro-Negra ou à segmentação de nodos da árvore B. Logo, para implementações onde a otimização é primordial, o uso da árvore AVL deve ser evitado.

## 6 REFERÊNCIAS BIBLIOGRÁFICAS

OLAWANLE, J. *Big O cheat sheet – time complexity chart*. **FreeCodeCamp**, 5 out. 2022.

Disponível em:

<https://www.freecodecamp.org/news/big-o-cheat-sheet-time-complexity-chart/>. Acesso em: 25 Nov. 2025

GEEKS FOR GEEKS. AVL Tree Data Structure. **GeeksForGeeks**, 11 out. 2025. Disponível em: <https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/>. Acesso em: 24 Nov. 2025

GEEKS FOR GEEKS. Introduction of B Trees. **GeeksForGeeks**, 30 Jul. 2025. Disponível em: <https://www.geeksforgeeks.org/dsa/introduction-of-b-tree-2/>. Acesso em: 25 Nov. 2025

GEEKS FOR GEEKS. Introduction to Red-Black Tree. **GeeksForGeeks**, 03 Nov. 2025. Disponível em: <https://www.geeksforgeeks.org/dsa/introduction-to-red-black-tree/>. Acesso em: 25 Nov. 2025